The Metatheory of Crush Defunctionalization

lean-crush

August 2026

Abstract

This document specifies the proof-facing higher-order language used by Crush, its first-order target, their model semantics, closure conversion by defunctionalization, and the semantic soundness theorem. It follows the Lean development and deliberately distinguishes the verified mathematical core from the refinement boundary to the live translator. The displayed notation agrees with the centralized Lean notation file.[1]

Contents

[1] Lean formalization: `Crush/Metatheory/Notation.lean`.

1 Overview and notation

Contents of this section. The verified pipeline; conventions; correspondence with Lean notation.

The source language is a simply typed higher-order logic with opaque base sorts. The target is many-sorted first-order logic. Defunctionalization replaces every function space by an opaque first-order sort, every lambda by a closure constructor, and every higher-order application by an application symbol.

Metavariable conventions

The following conventions apply before any formulas below are read:

e, f, a	terms (with f usually function-valued and a an argument),
φ, ψ	Boolean formulas; φ is closed when called a sentence,
β	an opaque source base sort,
σ, τ, υ	source types,
κ	a first-order target sort,
Γ	a local typing context,
Σ	a signature of uninterpreted constants or symbols,
$\mathcal{M}$	a semantic model,
ρ	a valuation assigning semantic values to local variables,
ν	an individual semantic value,
R	a bundled relation between a source model and a target model,
Δ	a target theory, that is, a list of closed formulas,
$\mathcal{D}$	the core syntax-to-syntax defunctionalization map,
$\mathcal{T}(\varphi)$	the complete target theory generated from φ.

Subscripts s and t mean *source* and *target*; for example, $\mathcal{M}_s$ and $\mathcal{M}_t$ are source and target models, while ρ_s and ρ_t are their valuations. A symbol without a subscript is used when the side is clear or the statement applies to either language. We use Greek metavariables where conventional, rather than program-style identifiers.

Definition 1.1 (Carrier)**.** *A* carrier *is the collection of semantic values that a model assigns to a logical sort. For example, if β is an opaque sort intended to represent entities, a model may choose some nonempty type $B(\beta)$ as its carrier; the variables and constants of sort β then denote elements of $B(\beta)$. Syntax specifies only the name and relationships of sorts, whereas a model chooses their carriers. We require carriers to be nonempty, following the standard many-sorted and SMT semantics.*

For a closed source formula φ, the central result has the shape

$$\mathrm{Unsat}_{\mathrm{FO}}\big(\mathcal{T}(\varphi)\big) \quad \Longrightarrow \quad \mathrm{Unsat}_{\mathrm{HO}}(\varphi).$$

This is the direction needed to justify reporting unsatisfiability after the translation.

We use the following notation throughout:

$[\![e]\!]_{\mathcal{M},\rho}$	denotation of e in $\mathcal{M}$ under ρ,
$\mathcal{M} \models \varphi$	satisfaction of a closed formula,
$\mathcal{M} \models^{\mathsf{T}} \Delta$	satisfaction of every sentence in a theory,
$\lfloor \tau \rfloor$	first-order erasure of a source type,
$\mathcal{D}[\![e]\!]$	core defunctionalization of syntax,
$\lfloor \Gamma \rfloor^{\star}$	pointwise erasure of a context,
$\nu_s \approx_{R,\tau} \nu_t$	logical relation on values,
$\rho_s \approx_R^{\mathsf{v}} \rho_t$	pointwise logical relation on valuations,
$\mathsf{FV}(e)$	duplicate-free free-variable positions of e.

For example, $[\![e]\!]_{\mathcal{M},\rho}$ should be read from the conventions above as: "the semantic value of term e, using the interpretations supplied by model $\mathcal{M}$ and the local-variable assignment ρ." The model interprets global constants and carriers; the valuation interprets only the locally bound variables in e.

2 Intrinsically typed higher-order source

Contents of this section. Types and signatures; typed references; terms and formulas.

2.1 Types, contexts, and signatures

Definition 2.1 (Source types)**.** *Let β range over opaque base-sort identifiers. Source types are generated by*

$$\tau ::= \mathsf{Bool} \mid \mathsf{Base}(\beta) \mid \tau \to \tau.$$

Arrow types are nondependent and associate to the right. A context is a list $\Gamma = [\tau_0, \ldots, \tau_{n-1}]$, with the most recently introduced binder at the head. A signature Σ is likewise a list of the types of uninterpreted constants.[2]

Typed de Bruijn references are judgments $i : \tau \in \Gamma$, inductively generated by

$$\frac{}{0 : \tau \in (\tau :: \Gamma)}\ \textsc{here} \qquad \frac{i : \tau \in \Gamma}{i+1 : \tau \in (\sigma :: \Gamma)}\ \textsc{there}.$$

The same representation indexes constants in Σ. Consequently, ill-typed variables and constants are not inhabitants of the syntax datatype.

[2] Lean formalization: `Crush/Metatheory/HO/Core.lean`.

2.2 Terms

Definition 2.2 (Intrinsic source syntax)**.** *The judgment* $\Sigma;\Gamma \vdash e : \tau$ *is generated by the following forms:*

$$\frac{i : \tau \in \Gamma}{\Sigma;\Gamma \vdash x_i : \tau}\ \textit{VAR} \qquad \frac{k : \tau \in \Sigma}{\Sigma;\Gamma \vdash c_k : \tau}\ \textit{CONST} \qquad \frac{b \in \{\top, \bot\}}{\Sigma;\Gamma \vdash b : \mathsf{Bool}}\ \textit{BOOL} \qquad \frac{\Sigma;\Gamma \vdash \varphi : \mathsf{Bool}}{\Sigma;\Gamma \vdash \neg\varphi : \mathsf{Bool}}\ \textit{NOT}$$

$$\frac{\Sigma;\Gamma \vdash \varphi : \mathsf{Bool} \qquad \Sigma;\Gamma \vdash \psi : \mathsf{Bool} \qquad \circ \in \{\wedge, \vee, \Rightarrow, \Leftrightarrow\}}{\Sigma;\Gamma \vdash \varphi \circ \psi : \mathsf{Bool}}\ \textit{CONNECTIVE}$$

$$\frac{\Sigma;\Gamma \vdash e_1 : \tau \qquad \Sigma;\Gamma \vdash e_2 : \tau}{\Sigma;\Gamma \vdash e_1 = e_2 : \mathsf{Bool}}\ \textit{EQ} \qquad \frac{\Sigma;\tau :: \Gamma \vdash e : \sigma}{\Sigma;\Gamma \vdash \lambda x{:}\tau.e : \tau \to \sigma}\ \textit{LAM}$$

$$\frac{\Sigma;\Gamma \vdash f : \tau \to \sigma \qquad \Sigma;\Gamma \vdash a : \tau}{\Sigma;\Gamma \vdash f\ a : \sigma}\ \textit{APP} \qquad \frac{\Sigma;\tau :: \Gamma \vdash \varphi : \mathsf{Bool} \qquad Q \in \{\forall, \exists\}}{\Sigma;\Gamma \vdash Qx{:}\tau.\varphi : \mathsf{Bool}}\ \textit{QUANT}\ .$$

A formula is a Boolean term, and a sentence is a formula in the empty context. The indices of Lean's `Term` *datatype enforce all premises of these rules.*[3]

Notice that quantification may range over arrow types. Thus "higher-order" describes both lambda/application and the possible domains of quantifiers.

3 Higher-order model semantics

Contents of this section. Interpretation of types; models and valuations; denotation and satisfaction.

Definition 3.1 (Type denotation)**.** *A family* B *assigns a nonempty carrier* $B(\beta)$*, represented by a Lean type, to each opaque base sort. Its extension to all source types is*

$$[\![\mathsf{Bool}]\!]_B = \mathsf{Prop}, \qquad [\![\mathsf{Base}(\beta)]\!]_B = B(\beta), \qquad [\![\tau \to \sigma]\!]_B = [\![\tau]\!]_B \to [\![\sigma]\!]_B.$$

[4]

Definition 3.2 (Source model and valuation)**.** *A source model* $\mathcal{M}_s$ *consists of nonempty carriers* $B(\beta)$ *and, for each* $k : \tau \in \Sigma$*, a value* $c_k^{\mathcal{M}_s} \in [\![\tau]\!]_B$*. A valuation* ρ_s *for* Γ *assigns to every typed reference* $i : \tau \in \Gamma$ *a value in* $[\![\tau]\!]_B$*. We write* $\rho_s[\nu]$ *for extension by a new head value* ν*.*

Definition 3.3 (Source denotation)**.** *The function* $[\![e]\!]_{\mathcal{M}_s,\rho_s}$ *is defined structurally. Variables are looked up in* ρ_s*, constants in* $\mathcal{M}_s$*, Boolean constructs have their ordinary propositional meanings, and equality is Lean equality. The distinctive clauses are*

$$\begin{aligned}
[\![\lambda x{:}\tau.e]\!]_{\mathcal{M}_s,\rho_s} &= \lambda\nu.[\![e]\!]_{\mathcal{M}_s,\rho_s[\nu]},\\
[\![f\ a]\!]_{\mathcal{M}_s,\rho_s} &= [\![f]\!]_{\mathcal{M}_s,\rho_s}([\![a]\!]_{\mathcal{M}_s,\rho_s}),\\
[\![\forall x{:}\tau.\varphi]\!]_{\mathcal{M}_s,\rho_s} &= \forall \nu \in [\![\tau]\!]_B, [\![\varphi]\!]_{\mathcal{M}_s,\rho_s[\nu]},\\
[\![\exists x{:}\tau.\varphi]\!]_{\mathcal{M}_s,\rho_s} &= \exists \nu \in [\![\tau]\!]_B, [\![\varphi]\!]_{\mathcal{M}_s,\rho_s[\nu]}.
\end{aligned}$$

[3] Lean formalization: `Crush/Metatheory/HO/Core.lean`.
[4] Lean formalization: `Crush/Metatheory/HO/Semantics.lean`.

For a sentence φ, $\mathcal{M}_s \models \varphi$ abbreviates evaluation under the empty valuation.[5]

Lemma 3.4 (Nonemptiness)**.** *If every $B(\beta)$ is nonempty, then $[\![\tau]\!]_B$ is nonempty for every τ.*

Proof. By induction on τ. At an arrow, choose a constant function returning an inhabitant of the codomain. □

4 Intrinsically typed first-order target

Contents of this section. First-order sorts and symbols; target syntax; target semantics.

4.1 Sorts and declarations

Definition 4.1 (First-order erasure)**.** *Target sorts are*

$$\kappa ::= \mathsf{Bool} \mid \mathsf{Base}(\beta) \mid \mathsf{Fn}(\tau, \sigma).$$

The erasure of a source type is

$$\lfloor \mathsf{Bool} \rfloor = \mathsf{Bool}, \qquad \lfloor \mathsf{Base}(\beta) \rfloor = \mathsf{Base}(\beta), \qquad \lfloor \tau \to \sigma \rfloor = \mathsf{Fn}(\tau, \sigma).$$

The last form is an opaque first-order sort, not a function space. Contexts are erased pointwise: $\lfloor [\tau_0, \ldots, \tau_n] \rfloor^\star = [\lfloor \tau_0 \rfloor, \ldots, \lfloor \tau_n \rfloor]$.[6]

A symbol declaration $\delta = (\bar{\kappa}; \kappa)$ consists of a list of argument sorts and one result sort. A target signature is a list of such declarations. Typed symbol references and heterogeneous argument vectors ensure that a symbol can only be applied at its declared arity and sorts.

Definition 4.2 (Target syntax)**.** *The first-order term judgment contains variables, symbol application, Boolean literals and connectives, same-sort equality, and universal and existential quantification. It contains neither lambda nor higher-order application. Function values occur only as elements of $\mathsf{Fn}(\tau, \sigma)$.* [7]

4.2 Target semantics

Definition 4.3 (Target model)**.** *Target carriers C provide nonempty types $C_B(\beta)$ and $C_F(\tau, \sigma)$. Sort denotation maps $\mathsf{Fn}(\tau, \sigma)$ to the arbitrary carrier $C_F(\tau, \sigma)$, rather than to a meta-level function space. A declaration $(\kappa_1, \ldots, \kappa_n; \kappa)$ denotes the curried type*

$$[\![\kappa_1]\!]_C \to \cdots \to [\![\kappa_n]\!]_C \to [\![\kappa]\!]_C.$$

A target model $\mathcal{M}_t$ chooses an interpretation of this type for every symbol. Term denotation, sentence satisfaction $\mathcal{M}_t \models \varphi$, and theory satisfaction $\mathcal{M}_t \models^{\mathsf{T}} \Delta$ are standard.[8]

The actual core translation uses a type-indexed family of symbols rather than a finite list while proving semantics. Family syntax and semantics differ only in how symbols are indexed; this avoids irrelevant finite-signature bookkeeping in the fundamental proof.[9]

[5] Lean formalization: `Crush/Metatheory/HO/Semantics.lean`.
[6] Lean formalization: `Crush/Metatheory/FO/Core.lean`.
[7] Lean formalization: `Crush/Metatheory/FO/Core.lean`.
[8] Lean formalization: `Crush/Metatheory/FO/Semantics.lean`.
[9] Lean formalization: `Crush/Metatheory/FO/Family.lean and FO/FamilySemantics.lean`.

5 Collection and generated declarations

Contents of this section. Free variables; closures and arrows; total collection; generated symbols.

Definition 5.1 (Free-variable positions)**.** $\mathsf{FV}(e)$ *is computed structurally as a left-to-right, duplicate-free list of de Bruijn positions. Under a binder, position* 0 *is removed and every* $i+1$ *becomes* i*. Binary forms concatenate their lists and erase duplicates.* [10]

Definition 5.2 (Closure descriptor)**.** *A closure descriptor is a tuple*

$$\chi = (\Gamma, \tau, \sigma, e) \quad \textit{where} \quad \Sigma; \tau :: \Gamma \vdash e : \sigma.$$

Its captures are $\mathsf{leave}(\mathsf{FV}(e))$*, hence are positions in* Γ*, not in* $\tau :: \Gamma$*. Total typed lookup converts these positions to an existentially packed list of typed references; their types form* $\mathsf{captureTypes}(\chi)$*.*

Definition 5.3 (Collection plan)**.** *An arrow key is a pair* (τ, σ) *representing* $\tau \to \sigma$*. The total collector recursively gathers all lambda occurrences and every arrow node in the source signature and term, removing duplicate arrow keys. Its result is*

$$P(e) = (\mathsf{arrows}(e), \mathsf{closures}(e)).$$

The collector is a total Lean function, and bounds and uniqueness lemmas show that every capture position names an actual context entry. [11]

The executable content of collection can be summarized independently of its Lean representation as follows. Here dedup preserves first-occurrence order, and $+\!+$ denotes list concatenation.

Procedure: $\mathsf{collect}(\Sigma, e)$

$\alpha \leftarrow \mathsf{dedup}(\mathsf{arrowsInSignature}(\Sigma) +\!+ \mathsf{arrowsInTerm}(e))$
$\kappa \leftarrow \mathsf{closures}(e)$
return (α, κ)

procedure $\mathsf{closures}(e)$
  match e **with**
    $x_i, c_k, b \Rightarrow []$
    $\lambda x{:}\tau.e' \Rightarrow [(\Gamma, \tau, \sigma, e')] +\!+ \mathsf{closures}(e')$
    $e_1\, e_2 \Rightarrow \mathsf{closures}(e_1) +\!+ \mathsf{closures}(e_2)$
    $\circ(e_1, \ldots, e_n) \Rightarrow \mathsf{concat}(\mathsf{map}(\mathsf{closures}, [e_1, \ldots, e_n]))$
  end match

For the classic proof core, each arrow key generates a unary symbol

$$\mathsf{app}_{\tau,\sigma} : \mathsf{Fn}(\tau, \sigma) \times \lfloor \tau \rfloor \longrightarrow \lfloor \sigma \rfloor,$$

and each $\chi = (\Gamma, \tau, \sigma, e)$ generates

$$\mathsf{clos}_\chi : \lfloor \gamma_{i_1} \rfloor \times \cdots \times \lfloor \gamma_{i_m} \rfloor \longrightarrow \mathsf{Fn}(\tau, \sigma),$$

where $i_1, \ldots, i_m$ are the exact captures of e.

[10] Lean formalization: `Crush/Metatheory/Defunctionalization/Collect.lean`.
[11] Lean formalization: `Crush/Metatheory/Defunctionalization/Collect.lean`.

6 Core defunctionalization

Contents of this section. Translation on syntax; closure equations; complete target theory.

Definition 6.1 (Core translation)**.** *The total, type-preserving translation satisfies*

$$\Sigma; \Gamma \vdash e : \tau \quad \Longrightarrow \quad \lfloor\Sigma\rfloor; \lfloor\Gamma\rfloor^{\star} \vdash \mathcal{D}[\![e]\!] : \lfloor\tau\rfloor.$$

Variables, constants, Boolean constructs, equality, and quantifiers translate homomorphically. Its essential clauses are

$$\mathcal{D}[\![\lambda x{:}\tau.e]\!] = \mathsf{clos}_{\chi}(x_{i_1}, \ldots, x_{i_m}),$$
$$\mathcal{D}[\![f\, a]\!] = \mathsf{app}_{\tau,\sigma}(\mathcal{D}[\![f]\!], \mathcal{D}[\![a]\!]),$$

where $\chi = (\Gamma, \tau, \sigma, e)$ and $i_1, \ldots, i_m$ are its captures. Source constants are nullary target symbols in this classic core. [12]

The complete syntax-directed procedure is shown below. The notation mapChildren means recursive, constructor-preserving traversal of the Boolean, equality, and quantifier forms.

Procedure: $\mathcal{D}[\![e]\!]$

---

match e **with**
  $x_i \Rightarrow x_{\lfloor i \rfloor}$
  $c_k \Rightarrow c_k^{\mathsf{FO}}()$
  $b \Rightarrow b$
  $\lambda x{:}\tau.e' \Rightarrow$
    $\chi \leftarrow (\Gamma, \tau, \sigma, e')$
    $(i_1, \ldots, i_m) \leftarrow \mathsf{captureRefs}(\chi)$
    return $\mathsf{clos}_{\chi}(x_{i_1}, \ldots, x_{i_m})$
  $e_1\, e_2 \Rightarrow \mathsf{app}_{\tau,\sigma}(\mathcal{D}[\![e_1]\!], \mathcal{D}[\![e_2]\!])$
  $\circ(e_1, \ldots, e_n) \Rightarrow \circ(\mathcal{D}[\![e_1]\!], \ldots, \mathcal{D}[\![e_n]\!])$
end match

In the final clause, $\circ$ ranges only over constructors retained by the target: Boolean literals and connectives, equality, and quantifiers. Thus the procedure is total and has no "unsupported syntax" case for the core language.

Definition 6.2 (Closure equation)**.** *For $\chi = (\Gamma, \tau, \sigma, e)$, let $\bar{x}$ enumerate all of Γ and let x denote the lambda argument. The defining equation is*

$$\forall \bar{x}\, x. \quad \mathsf{app}_{\tau,\sigma}(\mathsf{clos}_{\chi}(x_{i_1}, \ldots, x_{i_m}), x) = \mathcal{D}[\![e]\!].$$

Only captured variables are constructor arguments, although all variables from the original context are quantified. Let $\mathcal{E}(e)$ be the list of these equations for every lambda occurrence in e.

Definition 6.3 (Defunctionalization theory)**.** *For a source sentence φ,*

$$\mathcal{T}(\varphi) = \mathcal{E}(\varphi) +\!\!+ [\mathcal{D}[\![\varphi]\!]].$$

Thus a target model of $\mathcal{T}(\varphi)$ must satisfy both all closure equations and the translated query.[13]

[12] Lean formalization: `Crush/Metatheory/Defunctionalization/Core.lean`.
[13] Lean formalization: `Crush/Metatheory/Defunctionalization/ModelExtension.lean`.

7 The source–target logical relation

Contents of this section. Relation at each type; related valuations; compatibility of models.

Fix a source model $\mathcal{M}_s$, a target model $\mathcal{M}_t$, and relations $r_\beta \subseteq B(\beta) \times C_B(\beta)$ for opaque base sorts.

Definition 7.1 (Value relation)**.** *The relation $\nu_s \approx_{R,\tau} \nu_t$ is defined by recursion on τ:*

$$\begin{aligned} p \approx_{R,\mathsf{Bool}} q &\iff p \Leftrightarrow q, \\ a \approx_{R,\mathsf{Base}(\beta)} b &\iff r_\beta(a,b), \\ f \approx_{R,\tau\to\sigma} g &\iff \forall a, b.\ a \approx_{R,\tau} b \Rightarrow f(a) \approx_{R,\sigma} \mathsf{app}^{\mathcal{M}_t}_{\tau,\sigma}(g,b). \end{aligned}$$

The arrow clause is the semantic heart of defunctionalization: the opaque target value g behaves like f whenever it is observed through the generated application symbol.[14]

Definition 7.2 (Related valuations)**.** $\rho_s \approx^{\mathsf{v}}_R \rho_t$ *holds when every $i : \tau \in \Gamma$ satisfies*

$$\rho_s(i) \approx_{R,\tau} \rho_t(\lfloor i \rfloor).$$

Lemma 7.3 (Extension)**.** *If $\rho_s \approx^{\mathsf{v}}_R \rho_t$ and $\nu_s \approx_{R,\tau} \nu_t$, then $\rho_s[\nu_s] \approx^{\mathsf{v}}_R \rho_t[\nu_t]$.*

Proof. Case analysis on the typed de Bruijn reference: the head uses the new hypothesis, and a tail reference uses the old valuation relation. □

Definition 7.4 (Related models)**.** *A model relation $R : \mathcal{M}_s \sim \mathcal{M}_t$ packages the base relations and five compatibility obligations:*

(i) corresponding source and target constants are related;

(ii) equality of related source values is equivalent to equality of their target values;

(iii) the value relation is left-total;

(iv) the value relation is right-total;

(v) every translated closure constructor is related to the denotation of its source lambda under related valuations.

The two totality conditions are exactly what allow quantifiers in either model to be transported to the other.

8 Semantic preservation

Contents of this section. Fundamental lemma; sentence equivalence; canonical model extension; unsatisfiability soundness.

Theorem 8.1 (Fundamental lemma)**.** *Suppose $R : \mathcal{M}_s \sim \mathcal{M}_t$, $\Sigma; \Gamma \vdash e : \tau$, and $\rho_s \approx^{\mathsf{v}}_R \rho_t$. Then*

$$[\![e]\!]_{\mathcal{M}_s,\rho_s} \approx_{R,\tau} [\![\mathcal{D}[\![e]\!]]\!]_{\mathcal{M}_t,\rho_t}.$$

[15]

[14] Lean formalization: `Crush/Metatheory/Defunctionalization/LogicalRelation.lean`.
[15] Lean formalization: `Crush/Metatheory/Defunctionalization/Fundamental.lean`.

Proof structure. By induction on e. Variables use related valuations and constants use model compatibility. Boolean cases use propositional congruence. Equality uses equality compatibility. Lambda uses the closure obligation, and application unfolds the arrow clause of the logical relation. For $\forall$, right-totality turns an arbitrary target value into a related source value in one direction, and left-totality does the converse; $\exists$ uses the dual witness argument. □

Corollary 8.2 (Closed-formula preservation)**.** *For a sentence φ and related models,*

$$\mathcal{M}_s \models \varphi \quad \Longleftrightarrow \quad \mathcal{M}_t \models \mathcal{D}[\![\varphi]\!].$$

Proof. Empty valuations are related vacuously. Instantiate the fundamental lemma at Bool, where the logical relation is logical equivalence. □

8.1 Canonical target model

Given $\mathcal{M}_s$, the canonical model interprets target base sorts by source base carriers and target function sorts by a canonical representation of source functions. Its application symbols apply represented functions, source symbols use source interpretations, and closure constructors install their exact captured environment. The capture-agreement development proves that changing uncaptured entries cannot affect the body denotation.[16]

Theorem 8.3 (Closure validity)**.** *For every source model $\mathcal{M}_s$ and term e, its canonical target model $\widehat{\mathcal{M}_s}$ satisfies all collected closure equations:*

$$\widehat{\mathcal{M}_s} \models^{\mathsf{T}} \mathcal{E}(e).$$

Theorem 8.4 (Model extension)**.** *If $\mathcal{M}_s \models \varphi$, then*

$$\widehat{\mathcal{M}_s} \models^{\mathsf{T}} \mathcal{T}(\varphi).$$

[17]

Proof. Closure validity handles $\mathcal{E}(\varphi)$. The canonical models are related, so closed-formula preservation transports the truth of φ to $\mathcal{D}[\![\varphi]\!]$. □

Theorem 8.5 (Target unsatisfiability implies source unsatisfiability)**.** *For every closed source formula φ,*

$$\mathrm{Unsat}_{\mathrm{FO}}(\mathcal{T}(\varphi)) \Longrightarrow \mathrm{Unsat}_{\mathrm{HO}}(\varphi).$$

[18]

Proof. Assume a source model satisfies φ. Model extension constructs a target model satisfying $\mathcal{T}(\varphi)$, contradicting target unsatisfiability. □

[16] Lean formalization: `Crush/Metatheory/Defunctionalization/ModelExtension.lean`.
[17] Lean formalization: `Crush/Metatheory/Defunctionalization/ModelExtension.lean`.
[18] Lean formalization: `Crush/Metatheory/Defunctionalization/ModelExtension.lean`.

9 Eta expansion and flattened application

Contents of this section. Eta-long form; semantic eta correctness; production-style flattened symbols.

The production encoding uses one fully flattened symbol for a complete arrow spine. If

$$\tau = \tau_1 \to \cdots \to \tau_n \to \sigma$$

with non-arrow result σ, its declaration has the shape

$$\mathsf{app}_\tau : \mathsf{Fn}(\tau_1, \tau_2 \to \cdots \to \sigma) \times \lfloor \tau_1 \rfloor \times \cdots \times \lfloor \tau_n \rfloor \longrightarrow \lfloor \sigma \rfloor.$$

An under-applied symbol cannot represent a residual function, so partial applications must be converted to closures.

Definition 9.1 (Eta-long normalization)**.** *$\eta(e)$ recursively normalizes subterms. Whenever the reconstructed term has arrow type $\tau \to \sigma$ and is not already a lambda, it becomes*

$$\lambda x{:}\tau.\eta(e\, x),$$

recursing through σ as needed. Existing lambdas retain their outer binder.[19]

Procedure: $\eta(e)$

$e' \leftarrow \mathsf{mapChildren}(\eta, e)$
if $\mathsf{type}(e') = \tau \to \sigma$ **then**
  match e' **with**
    $\lambda x{:}\tau.e'' \Rightarrow \lambda x{:}\tau.e''$
    $\_ \Rightarrow \lambda x{:}\tau.\eta_\sigma(\mathsf{weaken}(e')\, x)$
  end match
else return e'

Lemma 9.2 (Arrow-value invariant)**.** *If $\Sigma; \Gamma \vdash e : \tau \to \sigma$, then $\eta(e)$ is syntactically a lambda.*

Theorem 9.3 (Eta semantic preservation)**.** *For every $\mathcal{M}_s$, ρ_s, and e,*

$$[\![\eta(e)]\!]_{\mathcal{M}_s,\rho_s} = [\![e]\!]_{\mathcal{M}_s,\rho_s}.$$

In particular, eta-expanding a partial application preserves its residual function value exactly.[20]

Theorem 9.4 (Flattened application preservation)**.** *Under the canonical interpretation of a production application symbol, feeding the translated argument spine to the single flattened symbol yields the same result as iterated source application.*[21]

Together, eta correctness and flattened-application preservation justify the production representation at the typed semantic layer. They are separate from the classic unary-core fundamental theorem; connecting concrete emitted syntax to these objects is a refinement obligation.

[19] Lean formalization: `Crush/Metatheory/Defunctionalization/Eta.lean`.
[20] Lean formalization: `Crush/Metatheory/Defunctionalization/EtaCorrectness.lean`.
[21] Lean formalization: `Crush/Metatheory/Defunctionalization/FlattenedApplication.lean`.

10 Guarded subtype encodings

Contents of this section. Encoding contract; guarded quantifiers and equality; natural numbers.

Definition 10.1 (Guarded encoding)**.** *A guarded encoding from S to T consists of nonempty S, a map $\epsilon : S \to T$, a predicate $G : T \to \mathsf{Prop}$, and a decoder $\delta : (t : T) \to G(t) \to S$ such that*

$$G(\epsilon(s)), \qquad \delta(\epsilon(s)) = s, \qquad G(t) \Rightarrow \epsilon(\delta(t)) = t.$$

Thus S is equivalent to the guarded subtype $\{t : T \mid G(t)\}$. [22]

Lemma 10.2 (Guarded quantifiers)**.** *If predicates $P : S \to \mathsf{Prop}$ and $Q : T \to \mathsf{Prop}$ agree on encoded values, then*

$$(\forall s, P(s)) \iff \forall t, G(t) \Rightarrow Q(t),$$
$$(\exists s, P(s)) \iff \exists t, G(t) \wedge Q(t).$$

Moreover $\epsilon(s_1) = \epsilon(s_2)$ iff $s_1 = s_2$.

The concrete natural-number encoding takes $S = \mathbb{N}$, $T = \mathbb{Z}$, $\epsilon(n) = n$, and $G(z) \equiv 0 \leq z$. The Lean development also lifts functions while enforcing guarded results and supports optional guarded fields.

11 Certified refinement to the live translator

Contents of this section. Typed bridges; capture and command certificates; extension hooks; trusted boundary.

The semantic theorem above speaks about proof-facing syntax. Production Crush starts with Lean expressions and emits named SMT commands. The bridge modules attach typed witnesses to the parts of that process covered by the core:

- type witnesses relate normalized, nondependent Lean arrows to source types and generated first-order declarations;[23]
- term and context bridges reify supported Lean expressions into intrinsic terms;[24]
- capture certificates connect the live free-variable collector to exact, duplicate-free typed closure captures;[25]
- command certificates retain the declarations and guarded equations that were actually emitted.[26]

Certified hooks state semantic preservation for restricted primitive mappings. The built-in Boolean negation exercises this path. Arbitrary metaprogram handlers remain an explicit trusted boundary rather than receiving a vacuous certificate.[27]

[22] Lean formalization: `Crush/Metatheory/Guarded/Encoding.lean`.
[23] Lean formalization: `Crush/Metatheory/Bridge/Type.lean`.
[24] Lean formalization: `Crush/Metatheory/Bridge/Term.lean and Bridge/Reify.lean`.
[25] Lean formalization: `Crush/Metatheory/Bridge/Capture.lean`.
[26] Lean formalization: `Crush/Metatheory/Bridge/Command.lean`.
[27] Lean formalization: `Crush/Metatheory/Hooks.lean`.

Remark 11.1 (Scope of the theorem). The complete theorem is established for every constructor of the modeled core. The live translator routes the supported fragment through proof-carrying bridge objects and falls back explicitly outside it. Therefore one should distinguish (i) the fully proved core semantic theorem, (ii) proved eta, flattening, guarded encoding, and hook components, and (iii) the remaining refinement assumptions for unsupported Lean expressions, recursive datatype well-formedness discovery, and unrestricted trusted handlers.

12 Dependency map and theorem inventory

Contents of this section. Proof dependency chain; principal machine-checked results.

Figure 1 gives the proof roadmap. Solid arrows are direct logical dependencies of the classic-core soundness theorem. Dashed arrows mark proved refinements used to relate that core result to production's flattened encoding; they are not premises of the unary-core fundamental lemma.

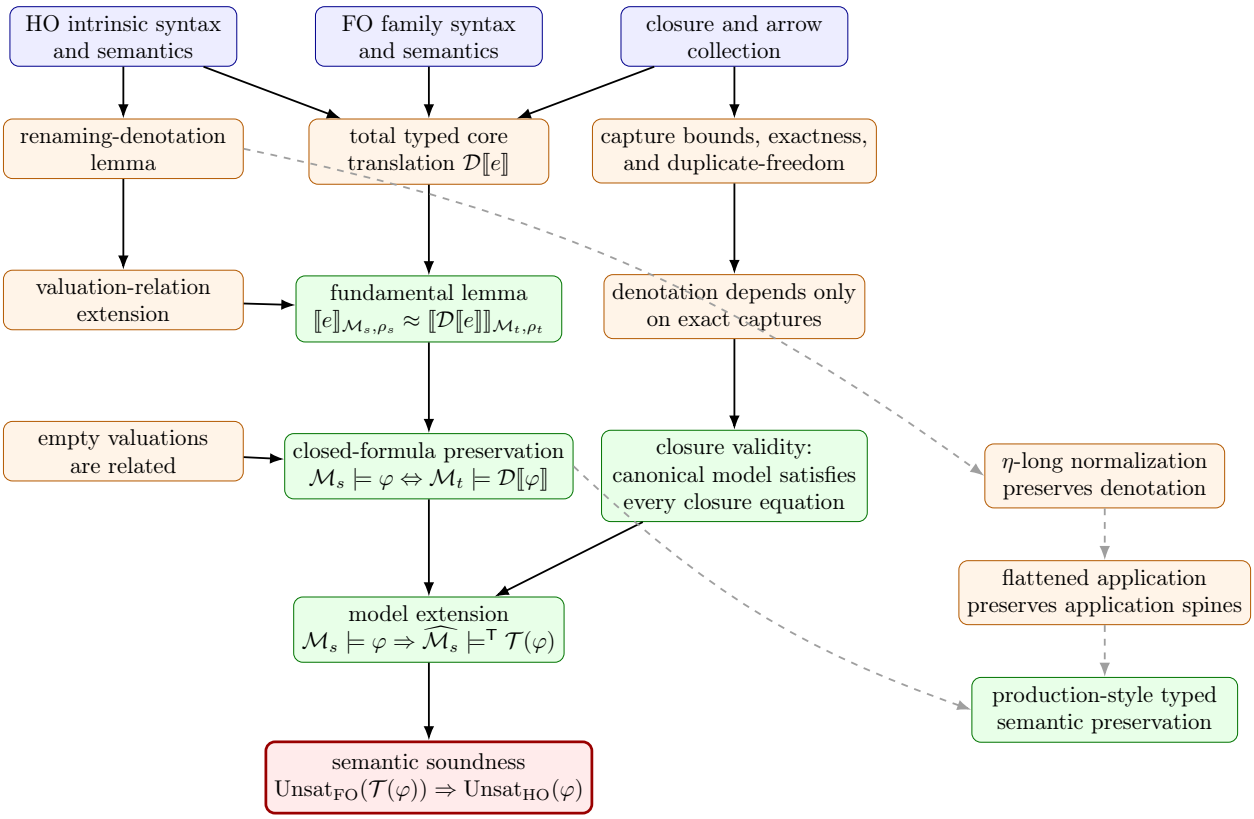

Figure 1: Dependency roadmap for the verified defunctionalization results. Solid arrows lead to classic-core semantic soundness; dashed arrows show the separate eta and flattened-application refinement toward the production encoding.

Reading the central solid path from top to bottom: intrinsic typing makes the translation total; the logical relation supports the fundamental induction; empty valuations specialize it to sentences; exact-capture reasoning validates the canonical closure equations; and these two branches meet in model extension, from which unsatisfiability soundness follows by contradiction.

The most important machine-checked results are:

1. typed capture positions are in bounds and duplicate-free;

2. $\eta(e)$ is a lambda at every arrow type and preserves denotation;
3. translated values of every source term are logically related;
4. closed source and translated formulas agree in related models;
5. the canonical model satisfies every generated closure equation;
6. every satisfying source model extends to a target model;
7. unsatisfiability of the generated target theory implies source unsatisfiability;
8. production-style flattened application preserves iterated application;
9. guarded universal, existential, equality, and result encodings are sound.